

TitleProject PassShield:

Local Password Validator (Web App Edition)

Project Name:

PassShield Password Validator

Objective:

To build a very essential utility tool which can directly run on a web browser with low ping or nearly zero network connection on the client side which could really and immediately tell if the password is strong or weak for security purposes.

Problem Statement:

Users need a quick, safe way to check if their passwords meet strong security criteria before using them in accounts. Using online password checkers risks sending sensitive credentials over the internet to unknown servers. A simple, standalone, local utility is needed to solve this privacy issue.

Research Findings:

Client-Side Validation: JavaScript regex (Regular Expressions) can instantly parse and evaluate text inputs directly inside the browser environment without sending data over a network.

Zero-Network Architecture: Running a local HTML file ensures complete sandbox isolation, meaning no password data ever leaves the physical mobile device. **Mobile-First Responsiveness:** Modern CSS layout viewports allow single-purpose utilities to adapt perfectly to touch screens of any size.

Methodology:

Streamlined UI: Having a simple and easy to use UI with a normal space to fill password and one button to verify its strength without any sharp colour.

Instant Feedback Logic: Write a JavaScript validation function that checks compliance rules in real-time upon a button press.

Clean Visual States: Use distinct color coding (red for weak, green for strong) to instantly communicate security feedback.

Tools Used:

Languages: HTML5, CSS3, JavaScript (ES6)

Core Logic Engine: JavaScript Regular Expressions (RegExp.test)

Environment: Any mobile browser (Google Chrome, Safari, Samsung Internet, Firefox)

Application: Spck Editor (for code implementation)

Implementation:

To make this project working and run on browser on the client side following file/code is required is as follow:-

Source code:-

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content
        ="width=device-width, initial
        -scale=1.0">
6      <title>PassShield Mobile</title>
7      <style>
8          :root {
9              --bg-color: #f4f7f6;
10             --card-bg: #ffffff;
11             --text-main: #333333;
12             --primary: #007bff;
13             --success: #28a745;
14             --danger: #dc3545;
15         }
16         body {
17             font-family: -apple-system
18                 , BlinkMacSystemFont,
19                 "Segoe UI", Roboto,
20                 sans-serif;
21             background-color: var(--bg
                -color);
22             color: var(--text-main);
23             margin: 0;
24             padding: 20px;
```

```
22     display: flex;
23     justify-content: center;
24     align-items: center;
25     min-height: 90vh;
26 }
27 .container {
28     width: 100%;
29     max-width: 450px;
30     background: var(--card-bg
31         );
32     padding: 25px;
33     border-radius: 16px;
34     box-shadow: 0 4px 12px
35         rgba(0,0,0,0.08);
36 }
37 h1 {
38     text-align: center;
39     font-size: 24px;
40     margin-bottom: 25px;
41     color: #111;
42 }
43 .section {
44     margin-bottom: 10px;
45 }
46 h2 {
47     font-size: 16px;
48     text-transform: uppercase;
49     letter-spacing: 0.5px;
```

```
48         color: #666;
49         margin-bottom: 12px;
50     }
51     input[type="password"] {
52         width: 100%;
53         padding: 14px;
54         margin-bottom: 15px;
55         border: 1px solid #ccc;
56         border-radius: 8px;
57         font-size: 16px;
58         box-sizing: border-box;
59     }
60     button {
61         width: 100%;
62         padding: 14px;
63         background-color: var
64             | (--primary);
65         color: white;
66         border: none;
67         border-radius: 8px;
68         font-size: 16px;
69         font-weight: bold;
70         cursor: pointer;
71         transition: background 0
72             | .2s;
73     }
74     button:active {
75         opacity: 0.9;
```

```
75         .result-box {
76             margin-top: 20px;
77             padding: 14px;
78             border-radius: 8px;
79             font-size: 15px;
80             font-weight: 500;
81             word-break: break-word;
82             display: none;
83         }
84     </style>
85 </head>
86 <body>
87
88 <div class="container">
89     <h1>🛡️ PassShield</h1>
90
91     <!-- PASSWORD VALIDATOR -->
92     <div class="section">
93         <h2>Password Validator</h2>
94         <input type="password" id
95             ="passwordInput"
96             placeholder="Enter
97             password to test...">
98         <button onclick="checkPassword
99             ()">Verify Strength
100         </button>
101         <div id="pwdResult" class
102             ="result-box"></div>
```



```
97     </div>
98 </div>
99
100 <script>
101     function checkPassword() {
102         const password = document
103             .getElementById
104             ('passwordInput').value;
105         const resultDiv = document
106             .getElementById
107             ('pwdResult');
108         resultDiv.style.display =
109             'block';
110
111         if (password.length === 0) {
112             resultDiv.innerHTML = "✗
113                 Error: Please enter a
114                 password first.";
115             resultDiv.style
116                 .backgroundColor =
117                 "#fde8e8";
118             resultDiv.style.color =
119                 "var(--danger)";
120             return;
121         }
122         if (password.length < 8) {
123             resultDiv.innerHTML = "✗
124                 Weak: Must be at least
```

```
114         8 characters long.";
resultDiv.style
        .backgroundColor =
        "#fde8e8";
115 resultDiv.style.color =
        "var(--danger)";
116 return;
117 }
118 if (!/[A-Z]/.test(password) ||
        !/[a-z]/.test(password)) {
119     resultDiv.innerHTML = "✗
        Weak: Must include
        both upper and
        lowercase letters.";
120 resultDiv.style
        .backgroundColor =
        "#fde8e8";
121 resultDiv.style.color =
        "var(--danger)";
122 return;
123 }
124 if (!/\d/.test(password) || !
        /[!@#$%^&*(),.?":{}|<>]/
        .test(password)) {
125     resultDiv.innerHTML = "✗
        Weak: Must include at
        least one number and
        one symbol.";
```



```
index.html
126     resultDiv.style
127         .backgroundColor =
128             "#fde8e8";
129     resultDiv.style.color =
130         "var(--danger)";
131     return;
132 }
133 resultDiv.innerHTML = "💪
134     Strong: Password meets
135     safety criteria!";
136 resultDiv.style
137     .backgroundColor =
138         "#eafaf1";
139 resultDiv.style.color = "var
140     (--success)";
141 }
142 </script>
143
144 </body>
145 </html>
```

Screenshots:

These are results of the following codes and the project:-



localhost:7700/index



Move address bar to the bottom

Copy link



PassShield

PASSWORD VALIDATOR

Enter password to test...

Verify Strength

This is the interface



localhost:7700/index



PassShield

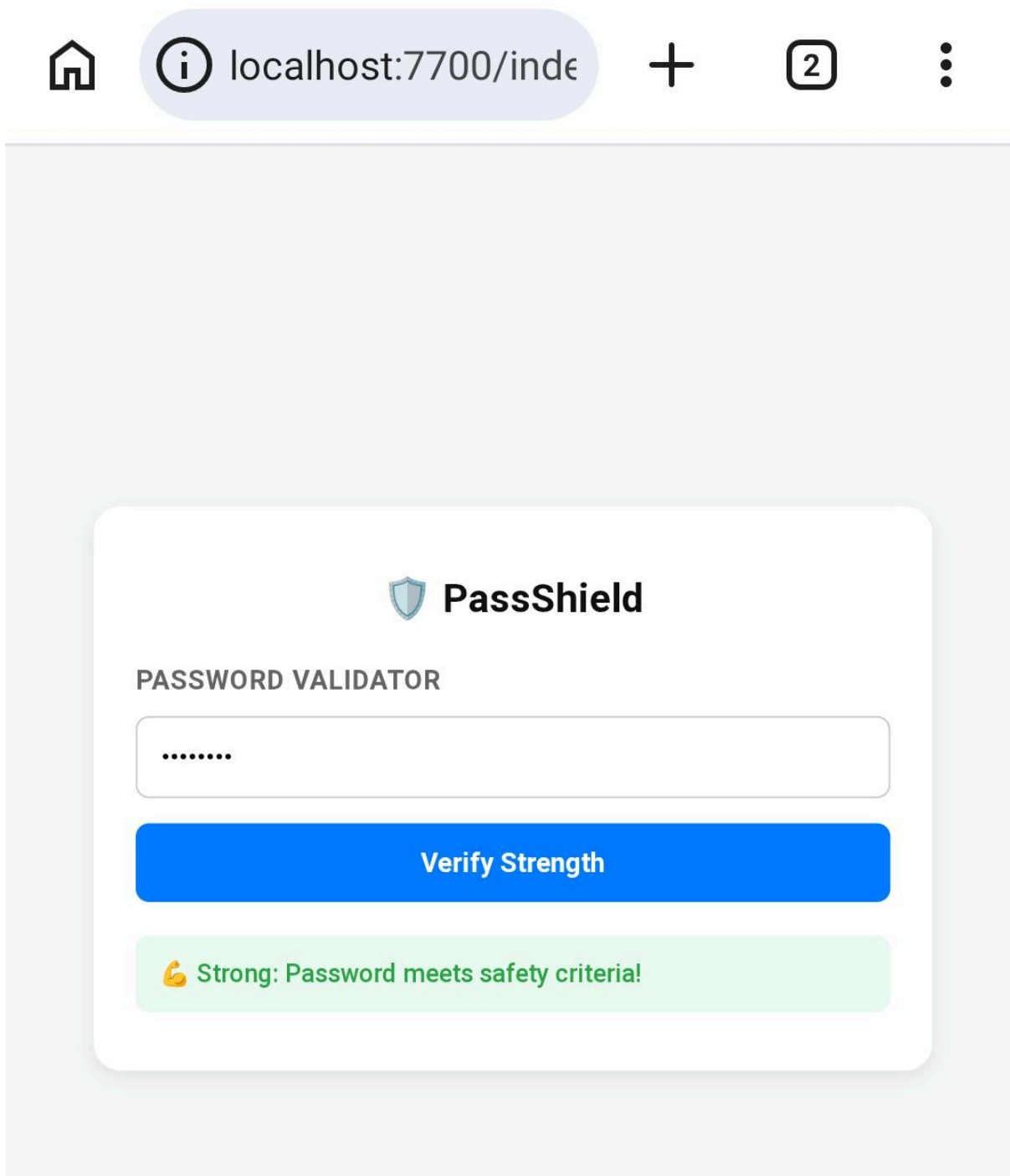
PASSWORD VALIDATOR

.....

Verify Strength

✗ Weak: Must include both upper and lowercase letters.

This image shows the weak password strength and the password which is used in it is hello123



This image show the strong password strength in this password used is Hello@123

Results:

Frictionless Workflow: The dashboard instantly profiles text inputs against 4 security rules simultaneously.

Absolute Confidentiality: Operates completely in the client-side browser memory layer, leaving no digital cache trail or network footprint.

Infinite Reusability: The single code asset can handle continuous back-to-back testing cycles without data rot or runtime performance

Challenges:

Input Clarity: Because characters are masked by default (type="password"), users may mistype their target strings without realization unless given a clear way to see their text.

Flaws: Due to little bit lack of in my skills it can't tell real time strength of password and also not able to show word which are return cause it is just a prototypes.

Future Scope:

Live Assessment: Upgrade event bindings from a button press to an oninput listener to score the security rating dynamically as the user types.

Entropy Metric: Implement a score counter to show character bits of entropy or estimated crack times alongside basic text flags.

Conclusion:

By limiting the architecture strictly to password scanning routines, PassShield becomes a highly targeted security utility. It completely eliminates technical deployment barriers while offering immediate privacy and protective calculations on mobile hardware.

This project is created by

Tushar

Student of Govt. Polytechnic Soniapt

nrshsaini11@gmail.com